DEVELOPER GUIDE

MCP Server

`Hryvinskyi_McpServer`

Install, connect, operate and extend the Model Context Protocol server of a Magento store.

MODULE VERSION	MAGENTO	PHP	PUBLISHED
1.5.0	2.4.6 – 2.4.9	8.1 – 8.5	2026-11-11

Magento Open Source and Adobe Commerce

© 2026 Hryvinskyi

Contents

1 **Start here** 3

2 **Installation** 4
- 2.1 Requirements 4
- 2.2 Install with Composer 4
- 2.3 Install by hand 4
- 2.4 Verify the installation 4
- 2.5 Upgrade 5

3 **First verified call** 6

4 **Concepts** 7
- 4.1 Abilities and the three tools 7
- 4.2 What decides whether a call runs 7
- 4.3 Callers 8
- 4.4 Confirmations 8
- 4.5 Personal data 8
- 4.6 GraphQL abilities 8

5 **Recipes** 9
- 5.1 Connect Claude Code 9
- 5.2 Configure a client by hand 9
- 5.3 Connect a client on the store's server (stdio) 9
- 5.4 Configure from the command line 10
- 5.5 Register an ability 10
- 5.6 Set the deny-list, confirmations and sections from code 12
- 5.7 Correct the access of a route 13
- 5.8 Extend the server in other ways 14

6 **Operations** 15

7 **Reference** 16
- 7.1 Settings 16
- 7.2 ACL resources 16
- 7.3 Endpoint and protocol 17
- 7.4 Limits 17
- 7.5 Responses and errors 17
- 7.6 Built-in deny-list 18
- 7.7 Service contracts 18
- 7.8 Configurable arguments 19

8 **Troubleshooting** 21

9 **Uninstall** 22

10 **Security checklist** 23

11 **Versioning** 24

Appendix A — Document history 25

1 Start here

MCP Server lets an MCP client, such as Claude Desktop, Claude Code, Cursor or VS Code, use a Magento Open Source or Adobe Commerce store through the Model Context Protocol (MCP). The client finds the operations the store already offers through its Web API (and, when enabled, its GraphQL schema), reads their arguments, and runs them as the integration or admin user it signs in as. Magento checks every call against that caller's ACL, as it does for any REST request.

This guide is for the developer who installs, configures, connects, operates or extends the module. The store admins' side (the admin screens, integrations for assistants, sections, the deny-list) is in the **User Guide**; this guide links to it rather than repeating it.

YOU WANT TO	READ
Install the module and check it runs	*Installation*
Make one real call and see the answer	*First verified call*
Understand what a client sees and why a call is refused	*Concepts*
Connect a client, configure from the CLI, add an ability, set the deny-list from code	*Recipes*
Run it in production	*Operations*
Look up a setting, limit, response or extension point	*Reference*
Fix a failure	*Troubleshooting*
Remove the module	*Uninstall*

Table 1. Where to go

2 Installation

2.1 Requirements

REQUIREMENT	VERSION
Magento Open Source or Adobe Commerce	2.4.6 – 2.4.9
PHP	8.1 – 8.5, with the `zlib` extension
`hryvinskyi/magento2-admin-ui` (module `Hryvinskyi_AdminUi`)	^1.0, installed by Composer with this module
`hryvinskyi/magento2-base`	^2.2 or ^3.0, installed by Composer
Cron	Magento's cron must run (session cleanup)

Table 2. Requirements

2.2 Install with Composer

BASH — Run from the Magento root

```
composer require hryvinskyi/magento2-mcp-server
bin/magento module:enable Hryvinskyi_Base Hryvinskyi_AdminUi Hryvinskyi_McpServer
bin/magento setup:upgrade
bin/magento setup:di:compile  # production mode only
bin/magento setup:static-content:deploy -f  # production mode only
bin/magento cache:flush
```

`module:enable` names the two modules this one requires as well: Magento refuses to enable a module while a module it requires is disabled. A module that is already enabled stays as it is.

`setup:upgrade` creates five tables (`hryvinskyi_mcp_session`, `hryvinskyi_mcp_confirmation`, `hryvinskyi_mcp_ability_override`, `hryvinskyi_mcp_section_off`, `hryvinskyi_mcp_personal_data_shown`) and enables the **MCP Server** cache type (`hryvinskyi_mcp`) once, through the data patch `EnableMcpCacheType`.

2.3 Install by hand

Place the module in `app/code/Hryvinskyi/McpServer`, require its Composer dependencies in the project (`hryvinskyi/magento2-admin-ui` and `hryvinskyi/magento2-base` among them), then run the commands above from `bin/magento module:enable` on.

2.4 Verify the installation

BASH — Run from the Magento root

```
bin/magento module:status Hryvinskyi_McpServer
```

Expected output:

TEXT

```
Hryvinskyi_McpServer : Module is enabled
```

The admin now has **Stores** > **Settings** > **MCP Server**. The server is **off** and **read-only mode is on** after installation: turning it on is the store's decision (*First verified call*, step 1).

2.5 Upgrade

Run `bin/magento setup:upgrade` after every upgrade, and in production mode `setup:di:compile` and `setup:static-content:deploy`. On Magento 2.4.9 also run `bin/magento cache:clean compiled_config` (or `cache:flush`): Magento caches the list of admin routes there, and until it is cleaned a route added by the upgrade answers the admin's 404 page. On the MCP Server page this shows as *The token could not be fetched*.

When an upgrade changes the Claude Desktop extension (its own version, see *Versioning*), users download the `.mcpb` file again and open it.

What an upgrade across these versions changes, from `CHANGELOG.md`:

UPGRADING PAST	WHAT CHANGES AND WHAT TO DO
1.1.0	GraphQL abilities appear, and **Enable GraphQL Abilities** is **Yes**. On a store where **Expose Anonymous Abilities** is already **Yes**, every GraphQL ability can run: set **Enable GraphQL Abilities** to **No** to keep the exposure of 1.0.x. The upgrade enables the **MCP Server** cache type once.
1.2.0	Requires `hryvinskyi/magento2-base` ^2.2 and `magento/module-ui`. `setup:upgrade` enables `Hryvinskyi_Base` when it is new.
1.3.0	Personal data is hidden from clients: the new **Hide Personal Data** starts at **Yes**, so results that carried names, e-mail addresses and the like now carry `[personal data]`. Where a data processing agreement covers it, show it for chosen abilities on the MCP Server page, or set **Hide Personal Data** to **No**.
1.4.0	Requires `hryvinskyi/magento2-admin-ui` (`Hryvinskyi_AdminUi`, enabled by `setup:upgrade`) and the `zlib` extension. On Magento 2.4.9 clean `compiled_config`, as above.

Table 3. Upgrade notes

3 First verified call

This chapter takes a store from installed to one real answer: the five most recently changed products.

1. Turn the server on: `bin/magento config:set hryvinskyi_mcp_server/general/enabled 1`, then `bin/magento cache:clean config`.
2. Let Magento accept integration tokens as Bearer tokens: `bin/magento config:set oauth/consumer/enable_integration_as_bearer 1`, then `bin/magento cache:clean config`.
3. In the admin, create and activate an integration whose **API** resources hold **Stores** > **MCP Server** > **Use the MCP endpoint** and **Catalog** > **Inventory** > **Products** (User Guide, *Create an integration for Claude*). Copy its **Access Token**.
4. Open a session (below), then call an ability.

Read the token without echoing it, so it stays out of the shell history:

BASH Run in your terminal

```
read -rs MAGENTO_MCP_TOKEN && export MAGENTO_MCP_TOKEN
curl -s -i https://<host>/rest/all/V1/mcp \
  -H "Authorization: Bearer $MAGENTO_MCP_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-06-18","capabilities":
{},"clientInfo":{"name":"check","version":"1.0"}}}'
```

`<host>` is the store's host name. Expected: HTTP status 200, a `mcp-session-id` header of 32 hexadecimal characters, and a body whose `result` holds `"protocolVersion":"2025-06-18"` and `"serverInfo":{"name":"Magento MCP Server","version":"1.5.0"}`.

Call an ability in that session, with the session id from the header:

BASH Run in your terminal

```
curl -s https://<host>/rest/all/V1/mcp \
  -H "Authorization: Bearer $MAGENTO_MCP_TOKEN" \
  -H "Content-Type: application/json" \
  -H "Mcp-Session-Id: <session-id>" \
  --data '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"name":"execute_ability","arguments":
{"name":"catalog.product_repository.get_list","arguments":{"searchCriteria":{"pageSize":5,"currentPage":1,"sortOrders":
[{"field":"updated_at","direction":"DESC"}]}}}}'
```

Expected: a `result` whose first `content` item is JSON text holding `items` (five products with their `sku` and `name`) and `total_count`. The most common failures of this first call:

- HTTP 401: the bearer setting of step 2 is off, the token is wrong, or the integration lacks **Use the MCP endpoint**.
- HTTP 403 with error `-32001`: the server is off (step 1).
- Error `-32003` with reason `acl`: the integration lacks **Products**.

4 Concepts

4.1 Abilities and the three tools

An *ability* is one operation a client may run. Abilities come from three sources, in this order of precedence:

SOURCE	WHAT IT OFFERS	NAME
`pool`	Abilities other modules register in `di.xml` (the module ships none)	chosen by the module
`webapi`	One ability per Web API service method (REST routes)	`<module>.<service>.<method>`, for example `catalog.product_repository.get_list`
`graphql`	One ability per root field of the store's GraphQL schema, plus `graphql.schema.describe_type`	`graphql.query.products`, `graphql.mutation.create_empty_cart`

Table 4. Ability sources

Abilities are not MCP tools themselves. The server offers three tools:

TOOL	WHAT IT DOES
`discover_abilities`	Searches abilities by words, `category` (a module name such as `Magento_Catalog`, or `GraphQL`) and `access` (`read`, `write`, `delete`); `limit` and `offset` page through them. It lists every ability, whatever the caller's role allows.
`get_ability_info`	Describes one ability: its arguments (JSON Schema), its ACL resources, whether the caller holds them, its exposure, and whether personal data is hidden.
`execute_ability`	Runs one ability with its `arguments`, and the `confirmationToken` when one was asked for. `fields` keeps only the named fields of the result, as dot-separated paths such as `items.sku`; a field the result lacks is named in a note after it.

Table 5. The three tools

The answer to `initialize` carries instructions for the model: how the three tools work together, that personal data arrives hidden while it does, and that timestamps such as `created_at` are UTC, with the time zone the store runs in (each store view's zone when they differ). The time zone is read from **General › Locale Options › Timezone** of each store view. The parts are the `parts` pool of `Hryvinskyi\McpServer\Model\Protocol\Instruction\ServerInstructions`.

4.2 What decides whether a call runs

Before an ability runs, the invoker checks, in this order: the exposure, the caller's ACL, a due confirmation, and the argument guards. A denied call changes nothing.

The exposure is the strictest answer of these rules:

ORDER	EXPOSURE	WHEN
1	**Denied**	On the built-in deny-list, on the page's deny-list, in a section switched off, or open to guests while **Expose Anonymous Abilities** is **No**.
2	**Read-only blocked**	**Read-only Mode** is **Yes** and the ability writes or deletes.
3	**Needs confirmation**	The ability requires a confirmation.
4	**Exposed**	None of the above.

Table 6. Exposure, strictest first

Access comes from the Web API route's HTTP method: `GET` reads, `POST` and `PUT` write, `DELETE` deletes. A route whose method tells the wrong story, such as a `GET` route that changes data, is corrected by name (*Correct the access of a route*).

4.3 Callers

A client signs in with an integration's access token (recommended: it does not expire, works with two-factor authentication and carries exactly the resources given to it) or with an admin token. A client on the store's own server can instead start `bin/magento hryvinskyi:mcp:serve`, which acts as an admin user named on the command line. Web API and pool abilities run in the admin store scope.

4.4 Confirmations

An ability that requires confirmation does not run on the first call. The server answers error `-32004` with a `confirmationToken`; the client asks the user, then calls again with the same name, the same arguments and that token. A token works once, for ten minutes, for the same session, caller, ability and arguments; an unknown, expired or spent token counts as none and the client gets a new one.

> ⚠ **WARNING**
> A confirmation token proves that the client made a second, explicit call for exactly these arguments. It does not prove that a person approved it. Use the deny-list or read-only mode for what must never run unattended.

4.5 Personal data

Every value under a field that **Personal Data Fields** lists reaches the client as `[personal data]`, however deep it sits. Field names match in a normalised form (lower case letters and digits only), so `*firstname` covers `firstname`, `customer_firstname` and `customerFirstname`. A custom attribute is judged by its code; `null` stays `null`; arguments are not touched. Free text, such as order comments, is not on the default list: add `comment` and `comments` to hide it. `initialize` tells the client that personal data arrives hidden; `get_ability_info` answers `personalData` (`hidden` or `shown`) per ability.

4.6 GraphQL abilities

Every query field of the store's GraphQL schema is an ability; mutations are offered when **Expose GraphQL Mutations** is **Yes**, and read-only mode blocks them. Every GraphQL ability runs as a guest: the server calls its own `/graphql` without a token or cookies, in the store view **Store View** names. So all of them stay denied until **Expose Anonymous Abilities** is **Yes**. Six query fields answer only a signed-in customer and are not offered: `customer`, `customerCart`, `customerDownloadableProducts`, `customerOrders`, `customerPaymentTokens` and `wishlist`.

5 Recipes

5.1 Connect Claude Code

The **Install** card of the MCP Server page shows the command, with the token in it once the **Access token** field holds one:

BASH — Run in your terminal

```
claude mcp add --transport http magento "https://<host>/rest/all/V1/mcp" --header "Authorization: Bearer <token>"
```

`<token>` is the integration's access token. The shell keeps the command, token included, in its history. To keep the token out of it, run `claude mcp add` without `--header` and add the header to the configuration file it writes.

5.2 Configure a client by hand

The **Manual configuration** part of the **Install** card shows each client's configuration with the file it goes in, and fills in the token once the **Access token** field holds one. Restart the client after saving the file.

Cursor (`.cursor/mcp.json` in a project, or `~/.cursor/mcp.json` for every project):

JSON — .cursor/mcp.json

```
{"mcpServers": {"magento": {"url": "https://<host>/rest/all/V1/mcp", "headers": {"Authorization": "Bearer <token>"}}}}
```

Claude Desktop without the extension (`claude_desktop_config.json`, opened from **Claude** > **Settings** > **Developer** > **Edit Config**): the file starts local commands only, so a remote server with a token goes through the `mcp-remote` bridge, which needs Node.js. Keep `Authorization:${MAGENTO_MCP_AUTH}` without a space:

JSON — claude_desktop_config.json

```
{"mcpServers": {"magento": {"command": "npx", "args": ["-y", "mcp-remote", "https://<host>/rest/all/V1/mcp", "--header",
"Authorization:${MAGENTO_MCP_AUTH}"], "env": {"MAGENTO_MCP_AUTH": "Bearer <token>"}}}}
```

The `.mcpb` extension the MCP Server page offers needs neither: it runs on the Node.js that ships with Claude Desktop, keeps the token in the system keychain, is named after the store's host so several stores install side by side, refuses an `http://` endpoint unless the host is `localhost`, `127.0.0.1` or `[::1]`, and never follows a redirect or retries a call.

5.3 Connect a client on the store's server (stdio)

A client on the same machine as the store can start a command instead of calling the endpoint. The command acts as the admin user you name:

BASH — Run from the Magento root

```
bin/magento hryvinskyi:mcp:serve --admin-user=<login>
```

`<login>` is the admin user's user name. Its role needs **Use the MCP endpoint** and the resources of the abilities it will run, and the server must be enabled. The **Manual configuration** part of the page shows the client's configuration with the store's path; add `-d display_errors=stderr`, so that a PHP error goes to standard error and never into the answers:

JSON — claude_desktop_config.json

```
{"mcpServers": {"magento": {"command": "php", "args": ["-d", "display_errors=stderr", "<path>/bin/magento",
"hryvinskyi:mcp:serve", "--admin-user", "<login>"]}}}
```

`<path>` is the Magento root. With DDEV, start the command in the container. If the client does not start in the project directory, add `"-p", "<project>"` after `"exec"`:

JSON claude_desktop_config.json

```
{"mcpServers": {"magento": {"command": "ddev", "args": ["exec", "php", "-d", "display_errors=stderr", "bin/magento",
"hryvinskyi:mcp:serve", "--admin-user", "<login>"]}}}
```

- Each message starts a new `bin/magento` process, about 0.4 s on a development machine; a changed setting, role or override reaches the next message without a restart.
- The command exits with code 1 when the admin user is missing, inactive, locked or expired, when the role lacks **Use the MCP endpoint**, or when the server is disabled.
- A message may run for 300 seconds; after that it is stopped and the client gets an error for it.

> ⚠ **WARNING**
> The stdio command checks no password and no token. Anyone who can run `bin/magento` on the server can act as any admin user through it. Name an admin whose role holds only what the client needs, and keep shell access limited.

5.4 Configure from the command line

`config:set` takes the stored values (`1` and `0`, not **Yes** and **No**). Clean the `config` cache after a change:

BASH Run from the Magento root

```
bin/magento config:set hryvinskyi_mcp_server/general/enabled 1
bin/magento config:set hryvinskyi_mcp_server/general/read_only_mode 1
bin/magento cache:clean config
```

All of the module's settings live in the default scope. Add `--lock-env` or `--lock-config` to keep a value in `app/etc/env.php` or `app/etc/config.php` instead of the database.

5.5 Register an ability

A module adds an ability by implementing `Hryvinskyi\McpServer\Api\PoolAbilityInterface` and adding the class to the `abilities` argument of `Hryvinskyi\McpServer\Model\Source\Pool\PoolAbilitySource`, keyed by the ability's name:

XML etc/di.xml

```
<type name="Hryvinskyi\McpServer\Model\Source\Pool\PoolAbilitySource">
    <arguments>
        <argument name="abilities" xsi:type="array">
            <item name="acme_shipping.rate_table.get"
xsi:type="object">Acme\Shipping\Model\Mcp\GetRateTableAbility</item>
        </argument>
    </arguments>
</type>
<type name="Acme\Shipping\Model\Mcp\GetRateTableAbility">
    <arguments>
        <argument name="rateTable" xsi:type="object">Acme\Shipping\Api\RateTableInterface\Proxy</argument>
    </arguments>
</type>
```

PHP Model/Mcp/GetRateTableAbility.php

```php
<?php
declare(strict_types=1);

namespace Acme\Shipping\Model\Mcp;

use Acme\Shipping\Api\RateTableInterface;
use Hryvinskyi\McpServer\Api\Data\AbilityAccess;
use Hryvinskyi\McpServer\Api\PoolAbilityInterface;
use Magento\Framework\Exception\LocalizedException;

class GetRateTableAbility implements PoolAbilityInterface
{
    public function __construct(private readonly RateTableInterface $rateTable)
    {
    }

    public function getName(): string
    {
        return 'acme_shipping.rate_table.get';
    }

    public function getTitle(): string
    {
        return 'Get the shipping rate table';
    }

    public function getDescription(): string
    {
        return 'Returns the rates of one carrier.';
    }

    public function getCategory(): string
    {
        return 'Acme_Shipping';
    }

    public function getAccess(): AbilityAccess
    {
        return AbilityAccess::Read;
    }

    public function getAclResources(): array
    {
        return ['Acme_Shipping::rates'];
    }

    public function getInputSchema(): array
    {
        return [
            'type' => 'object',
            'properties' => ['carrier' => ['type' => 'string']],
            'required' => ['carrier'],
        ];
    }

    public function execute(array $arguments): mixed
    {
        $carrier = $arguments['carrier'] ?? null;
        if (!is_string($carrier) || $carrier === '') {
            throw new LocalizedException(__('Name the carrier in the argument "carrier".'));
        }

        return $this->rateTable->forCarrier($carrier);
    }
}
```

Declare the resource in the module's `etc/acl.xml`:

XML etc/acl.xml

```xml
<?xml version="1.0"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="urn:magento:framework:Acl/etc/acl.xsd">
    <acl>
        <resources>
            <resource id="Magento_Backend::admin">
                <resource id="Magento_Backend::stores">
                    <resource id="Acme_Shipping::rates" title="Shipping Rate Tables" translate="title"/>
                </resource>
            </resource>
        </resources>
    </acl>
</config>
```

- The name follows `<module>.<service>.<method>` in snake case, at most 255 characters; title and category must not be blank.
- `getAclResources()` names at least one ACL resource, and the caller needs every one. An item with none is skipped: an empty list would let every caller through.
- Declare every resource it names. Magento allows an undeclared resource to every role that has all resources, and no other role can be given it. The resource's place in the tree also decides the ability's section on the MCP Server page.
- The access decides whether read-only mode blocks the ability. The exposure rules, overrides, confirmations and guards apply as to any ability; a registered ability wins over a Web API ability of the same name.
- `execute()` receives the client's arguments, as untrusted as any request body. Never pass them, or an array built from them, to a generated factory or to the object manager. A `LocalizedException` is shown to the client; outside developer mode any other exception is masked, and logged with a Report ID.
- An item that breaks a rule is skipped with a warning in the log, and the other abilities stay listed.
- Whenever the catalog is read, it calls the getters of every item, which builds the item. Keep the item cheap to build: inject a collaborator that is heavy to build as a `\Proxy`, as above, so that it is built only when `execute()` runs. A `\Proxy` on the item itself defers nothing.

Check it: clean the configuration cache (`bin/magento cache:clean config`; in production mode run `setup:di:compile` first). Then `discover_abilities` with the name as `query` lists it, and `get_ability_info` shows its schema.

5.6 Set the deny-list, confirmations and sections from code

The MCP Server page keeps its choices in the database. A data patch can make the same choices through the `@api` contracts, so that every environment starts with them:

PHP Setup/Patch/Data/RestrictMcpAbilities.php

```php
<?php
declare(strict_types=1);

namespace Acme\McpPolicy\Setup\Patch\Data;

use Hryvinskyi\McpServer\Api\AbilityOverrideRepositoryInterface;
use Hryvinskyi\McpServer\Api\Data\AbilityOverrideInterface;
use Hryvinskyi\McpServer\Api\Data\AbilityOverrideInterfaceFactory;
use Hryvinskyi\McpServer\Api\PersonalDataExceptionsInterface;
use Hryvinskyi\McpServer\Api\SectionSwitchesInterface;
use Magento\Framework\Setup\Patch\DataPatchInterface;

class RestrictMcpAbilities implements DataPatchInterface
{
    public function __construct(
        private readonly AbilityOverrideRepositoryInterface $overrides,
        private readonly AbilityOverrideInterfaceFactory $overrideFactory,
        private readonly SectionSwitchesInterface $sections,
        private readonly PersonalDataExceptionsInterface $personalData
    ) {
    }

    public function apply(): self
    {
        $this->overrides->save($this->overrideOf('catalog.product_repository.delete_by_id')->setDenied(true));
        $this->overrides->save($this->overrideOf('catalog.product_repository.save')->setNeedsConfirmation(true));
        $this->sections->switchOff('Magento_Customer::customer');
        $this->personalData->show(['sales.order_repository.get_list']);

        return $this;
    }

    public static function getDependencies(): array
    {
        return [];
    }

    public function getAliases(): array
    {
        return [];
    }

    private function overrideOf(string $ability): AbilityOverrideInterface
    {
        return $this->overrides->get($ability) ?? $this->overrideFactory->create()->setAbility($ability);
    }
}
```

- Ability names are the ones `discover_abilities` and the Ability catalog show. A section's code is the ID of its node in the ACL tree (`Magento_Customer::customer` for **Customers**), or `anonymous` (**Open to anyone**), `self` (**The caller's own account**) and `other` (**Other**).
- `save()` stores both flags of an override at once, and removes an override that neither denies nor asks for confirmation. Start from the stored one, as `overrideOf()` does, to keep the flag you do not change.
- A data patch runs once. An admin can change the result on the MCP Server page later; a later change in code needs a new patch.
- The patch's module declares `Hryvinskyi_McpServer` in its `module.xml` sequence and `hryvinskyi/magento2-mcp-server` in its `composer.json`.

5.7 Correct the access of a route

Read-only mode blocks an ability by its access, and a Web API ability takes its access from the route's HTTP method. For a route whose method tells the wrong story, such as a `GET` route that changes data, give the access by ability name:

XML etc/di.xml

```xml
<type name="Hryvinskyi\McpServer\Model\Source\Webapi\RouteAbilityBuilder">
    <arguments>
        <argument name="accessOverrides" xsi:type="array">
            <item name="acme_export.export_management.run" xsi:type="string">write</item>
        </argument>
    </arguments>
</type>
```

The value is `read`, `write` or `delete`. Any other value drops the ability from the catalog, with a line in the log that names it.

5.8 Extend the server in other ways

EXTENSION POINT	INTERFACE OR VALUE	`DI.XML` ARGUMENT
A source of abilities	`Hryvinskyi\McpServer\Api\AbilitySourceInterface`	`sources` of `Hryvinskyi\McpServer\Model\AbilityCatalog`, with a `sortOrder` (`pool` 10, `webapi` 20, `graphql` 30; a source without one sorts first)
An exposure rule	`Hryvinskyi\McpServer\Api\ExposureRuleInterface` (`ReasonedExposureRuleInterface` for its own denial reason)	`rules` of `Hryvinskyi\McpServer\Model\Exposure\ExposurePolicy`; a rule can only tighten
An argument guard	`Hryvinskyi\McpServer\Api\InvocationGuardInterface` (`refusal()` returns a message, or `null`)	`guards` of `Hryvinskyi\McpServer\Model\AbilityInvoker`
A result filter	`Hryvinskyi\McpServer\Api\ResultFilterInterface`	`resultFilters` of `Hryvinskyi\McpServer\Model\AbilityInvoker`
A call listener	`Hryvinskyi\McpServer\Api\InvocationListenerInterface`	`listeners` of `Hryvinskyi\McpServer\Model\AbilityInvoker`
More deny-list patterns	`fnmatch` patterns, keyed by reason	`patterns` of `Hryvinskyi\McpServer\Model\Exposure\Rule\BuiltInDenyListRule`
Web API services that are never abilities	Service interface names	`excludedServices` of `Hryvinskyi\McpServer\Model\Source\Webapi\RouteIndex` (the module's own endpoint is one)
GraphQL root fields that are never abilities	`<operation> <field>`, such as `query wishlist`	`excludedFields` of `Hryvinskyi\McpServer\Model\Source\GraphQl\GraphQlAbilitySource`

Table 7. Extension points

Configurable arguments lists the limits that `di.xml` can change.

6 Operations

- **Cron.** The job `hryvinskyi_mcp_session_cleanup` (group `default`, every hour) deletes expired sessions and confirmation tokens. Without cron, the session table only grows. To check that it runs, read its last runs from the database: `SELECT status, scheduled_at, executed_at FROM cron_schedule WHERE job_code = 'hryvinskyi_mcp_session_cleanup' ORDER BY scheduled_at DESC LIMIT 5;` shows a `success` row every hour.
- **Cache type.** Keep **MCP Server** (`hryvinskyi_mcp`) enabled; `bin/magento cache:status` lists `hryvinskyi_mcp: 1` while it is, and `bin/magento cache:enable hryvinskyi_mcp` turns it back on. It holds the Web API input schemas and the GraphQL schema; disabled, every request and every stdio message reads the whole GraphQL schema again, several MB through the loopback. After installing a module that changes the GraphQL schema, clean it: `bin/magento cache:clean hryvinskyi_mcp`.
- **Sessions.** Stored in the database: they survive a cache flush. A session expires 24 hours after its last use; a caller holds at most 20, and opening one more ends the least recently used.
- **PHP-FPM.** Each GraphQL call holds two workers for a moment (the call and the loopback). Keep `pm.max_children` above the number of clients that may call at the same time.
- **Errors.** The module keeps no log of calls. A failure with its exception goes to `var/log/exception.log`, a warning (such as an unreadable GraphQL schema) to `var/log/system.log`. A `LocalizedException` reaches the client as is. Outside developer mode, any other exception reaches it as *Internal Error. Details are available in Magento log file. Report ID: …*; look the Report ID up in `var/log/exception.log`. In developer mode the client gets the original message. A PHP error, such as a `TypeError`, reaches it as *Internal error* in every mode; the log holds it under *MCP ability "…" failed*.
- **HTTPS.** Serve the endpoint over HTTPS only: the token is a credential and every answer can hold store data.

7 Reference

7.1 Settings

All in **Stores** > **Settings** > **Configuration** > **MCP Server** > **MCP Server**, default scope only.

SETTING	PATH	DEFAULT	EFFECT
Enable MCP Server	`hryvinskyi_mcp_server/general/enabled`	`0`	While `0`, the endpoint refuses every message (HTTP 403, `-32001`).
Read-only Mode	`hryvinskyi_mcp_server/general/read_only_mode`	`1`	Abilities that write or delete are blocked.
Expose Anonymous Abilities	`hryvinskyi_mcp_server/general/expose_anonymous_abilities`	`0`	Lets clients run abilities whose only ACL resource is `anonymous` (guest checkout, account creation) and every GraphQL ability. Through MCP the CAPTCHA and rate limits of each REST route do not apply.
Hide Personal Data	`hryvinskyi_mcp_server/personal_data/hide`	`1`	Values under a personal field reach the client as `[personal data]`.
Personal Data Fields	`hryvinskyi_mcp_server/personal_data/fields`	email, names, phone, address, date of birth, tax and VAT number and similar	One field name per line; `*` stands for any characters.
Enable GraphQL Abilities	`hryvinskyi_mcp_server/graphql/enabled`	`1`	Offers every query field as an ability (denied until anonymous abilities are exposed).
Expose GraphQL Mutations	`hryvinskyi_mcp_server/graphql/expose_mutations`	`0`	Also offers the mutation fields.
GraphQL Endpoint URL	`hryvinskyi_mcp_server/graphql/endpoint_url`	empty	An absolute http(s) URL ending in `/graphql`, for a server that cannot reach its own public host name. Empty: the store view's secure base URL followed by `graphql`.
Store View	`hryvinskyi_mcp_server/graphql/store_code`	empty (**Default Store View**)	The store view GraphQL abilities run in, by its code; empty is the default store view.

Table 8. Settings

Integration tokens also need `oauth/consumer/enable_integration_as_bearer` = `1` (**Stores** > **Settings** > **Configuration** > **Services** > **OAuth** > **Consumer Settings** > **Allow OAuth Access Tokens to be used as standalone Bearer tokens**); it is `0` by default.

7.2 ACL resources

ROLE RESOURCE	ID	ALLOWS
Stores > **MCP Server**	`Hryvinskyi_McpServer::mcp_server`	The group of the three below
Stores > **MCP Server** > **Use the MCP endpoint**	`Hryvinskyi_McpServer::access`	Calling the endpoint and stdio. Every client needs it.
Stores > **MCP Server** > **View the MCP Server page**	`Hryvinskyi_McpServer::view`	The page, its catalog and the extension download
Stores > **MCP Server** > **Manage ability exposure**	`Hryvinskyi_McpServer::manage`	Deny-list, confirmations, sections and personal data on the page
Stores > **Settings** > **Configuration** > **MCP Server Settings**	`Hryvinskyi_McpServer::config`	The configuration section

Table 9. ACL resources

Picking an integration on the page also needs `Magento_Integration::integrations` (**System** > **System Extensions** > **System Integrations**). A caller's role is never used to hide an ability from the client; it decides only whether a call runs.

7.3 Endpoint and protocol

- `POST https://<host>/rest/all/V1/mcp` with a JSON-RPC 2.0 body and `Authorization: Bearer <token>`; `DELETE` ends the session. The store code must be `all`: `/rest/V1/mcp` and `/rest/default/V1/mcp` answer HTTP 400.
- Streamable HTTP with JSON responses, no event stream. Protocol versions `2025-11-25`, `2025-06-18`, `2025-03-26` and `2024-11-05`.
- Every message after `initialize` needs the `Mcp-Session-Id` header.
- A request whose `Origin` header names a host other than the store's base URLs is refused with HTTP 403.

7.4 Limits

LIMIT	VALUE	SET BY
Request body	1 MB	`maxBodyBytes`
Messages in a batch	20	`maxBatchSize`
JSON nesting depth	64 levels	fixed
Live sessions per caller	20	`maxSessionsPerCaller`
Session idle lifetime	24 hours	fixed
Confirmation token	10 minutes, one use	fixed
One stdio message	300 seconds, 1 MB	`messageTimeoutSeconds`, `maxMessageBytes`
GraphQL loopback call	Connects within 5 s, ends within 30 s, at most 5 MB, never follows a redirect	fixed

Table 10. Limits

Configurable arguments names the class of each argument.

7.5 Responses and errors

RESPONSE	CAUSE
HTTP 401, *The consumer isn't authorized to access Hryvinskyi_McpServer::access.*	No token, an invalid token, or a role without **Use the MCP endpoint**. Magento answers before the module runs.
HTTP 400, error `-32700`	*Parse error: the body is not valid JSON.*
HTTP 400, error `-32600`	The request as a whole is refused: a store code other than `all`; a body over 1 MB, nested deeper than 64 levels, or neither a JSON-RPC message nor a batch; an empty batch or one of more than 20 messages; a missing `Mcp-Session-Id` header; an `MCP-Protocol-Version` header the server does not speak or that differs from the session's. The message says which.
HTTP 403, error `-32001`	**Enable MCP Server** is **No**: *The MCP server is disabled.*
HTTP 403, error `-32600`	The `Origin` header names a host the store does not serve.
HTTP 404, error `-32600`	The session is unknown, expired or another caller's; the client sends `initialize` again.
HTTP 405, header `Allow: POST, DELETE`	Any other method, such as `GET`: the server offers no event stream.
Error `-32600`	One message is not valid JSON-RPC: a wrong `jsonrpc`, `method`, `params` or `id`. In a batch, only that message gets the error.
Error `-32601`	*Method not found*: a JSON-RPC method the server does not handle.
Error `-32602`	*Invalid params*: `arguments` is not a JSON object, or the tool does not exist (the message lists the tools).

RESPONSE	CAUSE
Error `-32603`	*Internal error*: the server failed. `var/log/exception.log` holds the exception, under *The MCP endpoint failed*, *MCP method "…" failed*, *The MCP session could not be checked* or, for stdio, *The MCP stdio message failed*.
Error `-32003`, reason `deny_list`	The ability is on a deny-list.
Error `-32003`, reason `read_only_mode`	Read-only mode blocks an ability that writes or deletes.
Error `-32003`, reason `anonymous_ability`	The ability is open to guests and anonymous abilities are not exposed.
Error `-32003`, reason `section_off`	A section of the ability is switched off.
Error `-32003`, reason `acl`	The caller's role lacks a resource; the message names it.
Error `-32003`, reason `argument_guard`	The arguments are refused, for example a customer save carrying a password hash.
Error `-32004`	The ability needs a confirmation (see *Confirmations*).
Tool result with `isError`	Unknown ability, wrong arguments, or the service failed. *This ability must be called over HTTPS.* for a route its module marks secure, called over plain HTTP. Outside developer mode, an unexpected exception reads *Internal Error. Details are available in Magento log file. Report ID: …*; a PHP error reads *Internal error*.

Table 11. Responses and errors

Errors without an HTTP status come with HTTP 200, inside the JSON-RPC answer.

7.6 Built-in deny-list

PATTERN	COVERS
`integration.*`	Integrations and the services that issue admin and customer tokens
`jwt_user_token.*`	JWT user tokens
`two_factor_auth.*`	Two-factor authentication
`login_as_customer*.*`	Login as customer
`customer.account_management.*password*`	Customer password reset and change
`graphql.mutation.*token*`	GraphQL mutations that issue, revoke or delete a token
`graphql.mutation.*password*`	GraphQL mutations that set or reset a password

Table 12. Built-in deny-list

7.7 Service contracts

The interfaces and enums under `Api/` and `Api/Data/` carry `@api`, Magento's mark for a contract other modules may rely on.

INTERFACE	WHAT IT IS FOR
`AbilityCatalogInterface`	Finds abilities: `get()`, `search()` (words, category, access, paging), `schemaOf()`, and `isAllowed()` for the current caller.
`AbilityInvokerInterface`	Runs one call through the exposure, ACL, confirmation and guard checks: `invoke()`.
`ExposurePolicyInterface`	The exposure of an ability, and the reason when a call is denied: `exposureOf()`, `decisionOf()`.
`ConfigInterface`	The three **General** settings: `isEnabled()`, `isReadOnlyMode()`, `isAnonymousExposed()`.
`PersonalDataPolicyInterface`	Whether an ability's results are masked: `isHidingOn()`, `hidesFor()`; `HIDDEN` is `[personal data]`.
`AbilityOverrideRepositoryInterface`	The deny-list and confirmations, per ability: `get()`, `save()`, `delete()`, `all()`.

INTERFACE	WHAT IT IS FOR
`SectionSwitchesInterface`	The sections switched off, by code: `isOff()`, `off()`, `switchOff()`, `switchOn()`.
`PersonalDataExceptionsInterface`	The abilities that show personal data while hiding is on: `shown()`, `show()`, `hide()`.
`McpEndpointInterface`	The Web API service the `/V1/mcp` routes are declared on. The MCP messages are answered before it is reached; its routes are never abilities.
`PoolAbilityInterface`, `AbilitySourceInterface`, `ExposureRuleInterface`, `ReasonedExposureRuleInterface`, `InvocationGuardInterface`, `ResultFilterInterface`, `InvocationListenerInterface`	Extension points: *Register an ability* and *Extend the server in other ways*.

Table 13. Service interfaces (`Hryvinskyi\McpServer\Api`)

TYPE	WHAT IT HOLDS
`AbilityInterface`	One ability, immutable: name (`NAME_PATTERN`, at most 255 characters), title, description, category, source and its reference, access, ACL resources.
`AbilityAccess`	Enum: `read`, `write`, `delete`.
`AbilityPageInterface`	One page of a search: `getTotal()`, `getItems()`.
`AbilityOverrideInterface`	One ability's stored override: on the deny-list, needs confirmation.
`Exposure`	Enum: `exposed`, `needs_confirmation`, `read_only_blocked`, `denied`; `stricter()` returns the stricter of two.
`ExposureDecisionInterface`	An exposure and, when a call is denied, its reason.
`InvocationInterface`	One call: ability name, arguments, whether it is confirmed, caller, client, transport, session id.
`CallerInterface`	The account whose ACL is evaluated: user type and user id.
`ClientInterface`	The MCP client's name and version, as it introduced itself.
`InvocationOutcome`	Enum: `succeeded`, `failed`, `denied`, `needs_confirmation`.
`InvocationResultInterface`	How a call ended: outcome, value, message, denial reason (`deny_list`, `read_only_mode`, `acl`, `anonymous_ability`, `argument_guard`, `section_off`), duration.

Table 14. Data types (`Hryvinskyi\McpServer\Api\Data`)

7.8 Configurable arguments

CLASS (`HRYVINSKYI\MCPSERVER\…`)	ARGUMENT	DEFAULT	WHAT IT SETS
`Model\Source\Webapi\RouteAbilityBuilder`	`accessOverrides`	empty	Access by ability name (*Correct the access of a route*)
`Model\Source\Webapi\RouteIndex`	`excludedServices`	`McpEndpointInterface`	Service interfaces whose routes are never abilities
`Model\Source\GraphQl\GraphQlAbilitySource`	`excludedFields`	the six customer-only query fields	Root fields never offered, as `<operation> <field>`
`Model\Protocol\Server`	`maxBodyBytes`	`1048576`	The longest request body, in bytes
`Model\Protocol\MessageParser`	`maxBatchSize`	`20`	Messages in one batch
`Model\Session\DbSessionStore`	`maxSessionsPerCaller`	`20`	Live sessions per caller
`Model\Protocol\ProtocolVersions`	`versions`	the four of *Endpoint and protocol*	Protocol versions offered, newest first

CLASS (`HRYVINSKYI\MCPSERVER\…`)	ARGUMENT	DEFAULT	WHAT IT SETS
`Model\Stdio\StdioLoop`	`messageTimeoutSeconds`, `maxLineBytes`	`300`, `1048576`	How long one stdio message may run; the longest line read from the client
`Model\Stdio\MessageHandler`	`maxMessageBytes`	`1048576`	The longest message one stdio process reads
`Model\Protocol\Instruction\ServerInstructions`	`parts`	`abilityWorkflow`, `personalData`, `timeZone`	The parts of the instructions `initialize` gives, in order

Table 15. Configurable arguments

These classes are outside `Api/`: read `CHANGELOG.md` before an upgrade if you change their arguments.

8 Troubleshooting

SYMPTOM	CAUSE AND FIX
Every client gets HTTP 401 with an integration token	`oauth/consumer/enable_integration_as_bearer` is `0`, the integration is not active, or its resources lack **Use the MCP endpoint**.
HTTP 403, `-32001`	The server is off: set `hryvinskyi_mcp_server/general/enabled` to `1` and clean the `config` cache.
`-32003` with `acl`	Add the resource the message names to the integration or role. `get_ability_info` lists an ability's resources.
The page's integration picker says the token could not be fetched	After an upgrade on 2.4.9, run `bin/magento cache:clean compiled_config`. Otherwise reload the page and pick again, or paste the token by hand, as the message says.
No GraphQL ability is listed	`var/log/system.log` says *MCP offers no GraphQL abilities: the GraphQL schema could not be read;* and why. Check **Enable GraphQL Abilities**; the server must reach its own `/graphql` within the loopback's limits (connect in 5 s, answer in 30 s, at most 5 MB, no redirects: a base URL that redirects fails). Set **GraphQL Endpoint URL** when the host does not resolve from the server, or the base URL is a placeholder such as `{{base_url}}`. Introspection must be allowed (`graphql/disable_introspection` not set in `app/etc/env.php`). Then clean the `hryvinskyi_mcp` cache. A failed schema read is remembered for five minutes.
GraphQL abilities are listed but denied	They run as a guest: set **Expose Anonymous Abilities** to **Yes**.
The `.mcpb` download fails with a message about the endpoint	The default store view's secure base URL must be an https address with a host name, without a user name or password.
Claude Desktop refuses the endpoint	The extension requires `https://` except on `localhost`, `127.0.0.1` or `[::1]`.
stdio exits with code 1 at start	The admin user is missing, inactive, locked or expired, lacks **Use the MCP endpoint**, or the server is off.
Error `-32603`, *Internal error*	`var/log/exception.log` holds the exception; *Responses and errors* lists the lines it is logged under.
A tool result reads *Internal Error. … Report ID: …*	Look the Report ID up in `var/log/exception.log`.
A tool result reads *Internal error*	A PHP error in the ability: `var/log/exception.log` holds it under *MCP ability "…" failed*.
A tool result reads *This ability must be called over HTTPS.*	Its route is marked secure in its module's `webapi.xml`: call the endpoint over HTTPS.

Table 16. Troubleshooting

9 Uninstall

The module ships no uninstall script. Removing it leaves its data behind; remove that data by hand if it is no longer needed.

BASH — Run from the Magento root

```
composer remove hryvinskyi/magento2-mcp-server
bin/magento setup:upgrade
bin/magento setup:di:compile  # production mode only
bin/magento cache:flush
```

What stays afterwards:

WHAT	WHERE
Five tables	`hryvinskyi_mcp_session`, `hryvinskyi_mcp_confirmation`, `hryvinskyi_mcp_ability_override`, `hryvinskyi_mcp_section_off`, `hryvinskyi_mcp_personal_data_shown`
Settings	`core_config_data` rows whose `path` starts with `hryvinskyi_mcp_server/`
Data patch record	`patch_list` row for `Hryvinskyi\McpServer\Setup\Patch\Data\EnableMcpCacheType`
Cache type switch	`hryvinskyi_mcp` under `cache_types` in `app/etc/env.php`
Integrations made for clients	**System** > **Extensions** > **Integrations**; delete them to revoke their tokens
Bearer tokens setting	`oauth/consumer/enable_integration_as_bearer` stays `1` if *First verified call* set it, so every integration's token still works as a standalone Bearer token. When no other integration needs that, run `bin/magento config:set oauth/consumer/enable_integration_as_bearer 0`.

Table 17. What uninstalling leaves

> ⚠ **WARNING**
> Back up the database before dropping tables or deleting rows. Dropping the five tables ends every MCP session and deletes the page's deny-list, confirmations, section switches and personal data exceptions.

10 Security checklist

- Serve the store over HTTPS.
- Create one integration per client, with **Use the MCP endpoint** and only the resources that client needs.
- Keep read-only mode on until a client needs to write, and then give its token narrow write resources.
- Leave **Expose Anonymous Abilities** off unless you accept guest operations without CAPTCHA and rate limits.
- Put abilities that must never run on the deny-list, or switch their section off.
- Keep personal data hidden unless a data processing agreement with the AI provider covers it.
- For stdio, use an admin user with a narrow role, and limit shell access to the server.
- Delete an integration when a client or person no longer needs it: its token stops working at once.
- After removing the module, set `oauth/consumer/enable_integration_as_bearer` back to `0` unless another integration needs it.

11 Versioning

The package follows semantic versioning, as `CHANGELOG.md` states: a patch release fixes, a minor release adds, a major release may break. The changelog lists every release; *Upgrade* sums up the ones that need an action.

- The `@api` contracts of *Service contracts* are what other modules may rely on. Classes outside `Api/` carry no such mark: 1.4.1 changed `IntegrationDirectoryInterface`, under `Model/Admin/Integration`, in a patch release. The `di.xml` arguments of this guide belong to such classes.
- No deprecation period is published beyond semantic versioning. Read the changelog before a major upgrade.
- The Claude Desktop extension has its own version (1.0.0), raised only when the bridge or its manifest changes; the package version does not change it.

Appendix A — Document history

VERSION	DATE	CHANGES
1.0	2026-11-09	First version, for MCP Server 1.4.2. Replaces the earlier single user manual, now split into this guide and the User Guide.
1.1	2026-11-11	For MCP Server 1.5.0: the instructions of `initialize` and their time zone, and `fields` of `execute_ability`.

Table 18. Document history